

COMP 208

Fall Semester 2025

INSTRUCTORS: Professor Michael Langer and DR. Chad Zammar

Assignment 1: Exploring Functions and Basic Python.

Posted: 16 September 2025 [last updated Sept 24]

Due: 30 September 2025, 11:59 PM.

Before you start:

- Use your IDE (Thonny) to write and debug your solution. Only use Ed Lessons to submit the code.
- Use only Python material covered in the videos or in class exercises for lectures 1 to 6, unless otherwise specified.
- If you get stuck, please see a T.A. or an instructor during office hours. When you come to office hours, please do not ask something like “Here is my program. What’s wrong with it?” Instead, try first to narrow down where the error is occurring and be ready to explain roughly what the problem is. We strongly encourage you to try to use the debugger.
- Policy on Academic Integrity: The assignments are to be done individually. While we encourage you to discuss the assignment with each other verbally, you should not tell each other how to solve the problem, and under no circumstances should you share your code or use Generative AI tools to generate it. See the Course Outline Sec. 6 for the policies. (You are expected to read that document.) If you have any questions, then please ask.
- Late policy: $k \cdot 10$ points out of 100 will be deducted if you submit k days of delay. So one day late means 10 points, two days late means 20 points, etc, up to 4 days late which is 40 points. Assignments will not be accepted after 4 days late.

-
- Note that submitting one minute late is equivalent to submitting 23 hours and 59 minutes late, etc. So, make sure you are nowhere near that threshold when you submit.

Assignment 1 learning objectives

- Using Thonny to program with Python
- Create and use variables
- Perform basic mathematical calculations with Python
- Handle user input and program output
- Use Python functions and create your own
- Use Python conditionals

The Problem

In this assignment, you will design and implement a Sleep Quality Assistant Program using Python. The program will interact with users to calculate their sleep quality score and weighted sleep quality Index. It will also provide personalized recommendations for improving their sleep habits.

Weighted Sleep Quality Index Calculations

Calculating the Weighted Sleep Quality Index involves assessing various factors such as average sleep time, sleep interruptions, and sleep environment.

Tasks:

You must implement the following functions.

main:

You must implement the function that gathers the user's input, calculates the sleep quality score based on this input, and also calculates the weighted sleep quality Index. The main function will have the following signature:

main()

This function should use helper functions to break down the tasks into multiple more manageable and smaller functions which are defined in the rest of this section.

Note: when you are writing and debugging your program in Thonny, your program will call the main() function. However, when you submit your assignment, we need you to comment out the line where you call main(). If you do not comment it out, then Ed Lessons will run main() and it will ask for the various inputs, just as when you run your code in Thonny. However, it will not run the tests. To run the tests, your submission can only include function definitions, but not function calls.

The helper functions called within main() are the following:

get_user_details:

This function aims to gather essential information by prompting the user to input certain details. The prompt should ask the user to enter each of the following:

- "Enter your average sleep time per night in hours: "
- "Enter the number of times you wake up during the night: "
- "Rate your sleep environment quality on a scale from 1 (poor) to 10 (excellent): "
- "Enter the average number of cups of coffee or caffeinated drinks you consume per day (0-10): "
- "Enter the average number of minutes of exercise per day (0-200): "
- "Rate your stress level on a scale from 1 (low) to 10 (high): "

You can assume that the values to be provided represent the daily average for the previous week. Assume also that the entered values are valid. The types should be as follows:

- ***Average Sleep time per night in hours: float***
- ***Number of times you wake up during the night: int***
- ***Sleep environment quality: int***
- ***Average number of cups of coffee: int***
- ***Minutes of exercise: int***

-
- ***Stress level: int***

The function should return all the values gathered in this order (average_sleep_time, sleep_interruptions, sleep_environment_quality, caffeine_consumption, exercise_duration, stress_level), and it has the following signature:

get_user_details()

calculate_sleep_quality_score:

This function calculates the sleep quality score based on the average sleep time and sleep interruptions. Use the following simple formula for calculating a sleep quality score:

Sleep Quality Score = ((Average Sleep Time in hours – Count of Sleep Interruptions) / 8) × 100

where:

Average Sleep Time: Total hours of sleep per night.

Sleep Interruptions: Number of times the user wakes up during the night

Please note that if the Sleep Quality Score is negative (i.e., the number of sleep interruptions exceeds average sleep time in hours), set the Sleep Quality Score to 0.

The function should return the sleep quality score, and it has the following signature:

calculate_sleep_quality_score(average_sleep_time, sleep_interruptions)

calculate_weighted_sleep_quality_index:

This function calculates the Weighted Sleep Quality Index (WSQI) based on the sleep quality score and other factors. The WSQI will be calculated using the following formula, except for one special case mentioned below.

$$\text{WSQI} = (0.5 \times \text{Sleep Quality Score}) + (0.3 \times \text{Sleep Environment Quality}) + (0.2 \times (10 - \text{Caffeine Consumption})) + (0.1 \times (\text{Minutes of Exercise}) / 15) - (0.1 \times \text{Stress Level})$$

The one special case is that if the Sleep Quality Score is 0 then set the WSQI to 0 also.

The function should round the result of the WSQI calculation to the nearest integer, and return the calculated wsqi.

The function should have the following signature:

```
calculate_weighted_sleep_quality_index(sleep_quality_score, sleep_environment_quality,
caffeine_consumption, exercise_duration, stress_level)
```

get_sleep_quality_group:

This function displays the group that the user belongs to according to their WSQI (poor, fair, good, excellent) using the following chart:

- Poor: 0 - 30
- Fair: 31 - 50
- Good: 51 - 70
- Excellent: 71- 100

Please note that the output to be returned from the function should match exactly one of the following:

- Poor
- Fair
- Good
- Excellent

In your main() function, make sure that the information is displayed to the user in a sentence like this one:

"Your Weighted Sleep Quality Index (WSQI) is *wsqi_value* and is considered *group_value*."

Example output:

Your Weighted Sleep Quality Index (WSQI) is 38 and is considered Fair.

If WSQI is "Poor", the program should generate concise, personalized recommendations based on the user's inputs as follows, and in the order given:

- **Begin with "Please consider the following recommendations:" [ADDED: Sept 18]**
- If average sleep time < 7 hours: tell the user how many additional hours per night are needed to reach 8 hours.
 - The phrasing should be: "Increase nightly sleep by X hour(s) to reach 8 hours."
- If the user wakes up during the night (sleep_interruptions > 0): recommend improving the sleep environment to reduce disruptions.
 - The phrasing should be: "Reduce nighttime awakenings by optimizing your bedroom (make it darker, cooler, and quieter) and limiting fluids/alcohol before bed."
- If caffeine >= 3 cups/day: tell the user how many cups to cut to fall below 3 cups and recommend a caffeine cut-off time.
 - The phrasing should be: "Reduce caffeine by X cup(s) to get below 3 cups/day, and avoid caffeine after mid-afternoon."
- If physical activity minutes per day < 15: tell the user how many additional minutes per day to add to reach 15 minutes, and suggest doing it earlier in the day.
 - Example phrasing: "Increase daily exercise by X minute(s) (ideally earlier in the day) to reach or exceed 15 minutes."

The function should have the following signature:

```
get_sleep_quality_group(wsqi)
```

Here are some examples of running the program. Your program must output the same as the following:

[Updated: Sept 24]

Example 1:

Enter your average sleep time per night in hours: 7.5
Enter the number of times you **wake** up during the night: 2
Rate your sleep environment quality on a scale from 1 (poor) to 10 (excellent): 8
Enter the average number of cups of coffee or caffeinated drinks you consume per day (0-10): 2
Enter the average number of minutes of exercise per day (0-200): 20
Rate your stress level on a scale from 1 (low) to 10 (high): 5
Your Weighted Sleep Quality Index (WSQI) is 38 and is considered Fair.

Example 2:

Enter your average sleep time **per night** in hours: 4.0
Enter the number of times you **wake** up during the night: 5
Rate your sleep environment quality on a scale from 1 (poor) to 10 (excellent): 3
Enter the average number of cups of coffee or caffeinated drinks you consume per day (0-10): 5
Enter the **average** number of minutes **of** exercise per day (**0-200**): 5
Rate your stress level on a scale from 1 (low) to 10 (high): 8
Your Weighted Sleep Quality Index (WSQI) is 0 and is considered Poor.
Please consider the following recommendations:
Increase nightly sleep by 4.0 hour(s) to reach 8 hours.
Reduce nighttime awakenings by optimizing your bedroom (make it darker, cooler, and quieter) and limiting fluids/alcohol before bed.
Reduce caffeine by 3 cup(s) to get below 3 cups/day, and avoid caffeine after mid-afternoon.
Increase daily exercise by 10 minute(s) (ideally earlier in the day) to reach or exceed 15 minutes.

The overall structure of your program should look like this:

Note that in this assignment you are not required to use docstrings. But you can put them in, if you wish.

```
def main():  
    """  
    """  
  
def get_user_details():  
    """  
    """
```

```
def calculate_sleep_quality_score(average_sleep_time, sleep_interruptions):
    """
    """

def calculate_weighted_sleep_quality_index(sleep_quality_score, \
    sleep_environment_quality, caffeine_consumption, \
    exercise_duration, stress_level):
    """
    """

def get_sleep_quality_group(wsqi):
    """
    """
```

Submission on Ed Lessons

Please read the instructions on Ed Lessons Assignment 1. Also follow these guidelines:

- Include your name and student ID in a comment at the top of the file along with a comment that says: "By submitting this file, I declare that I did the assignment on my own according to the rules specified in the assignment PDF."
- You may submit a README.txt file where you can tell the TA about any issues you ran into while doing this assignment.
- Comment out the call to main() as specified earlier in this document.
- We will provide several exposed (public) tests on Ed. If you pass all these tests then you are likely on the right track. These public tests will count for part of your grade. However, there will also be private tests which may consider more subtle cases.

We strongly encourage you to come up with more creative test cases, and to share these test cases on the discussion board.

- You may submit as many times as you like, though only the last submission will be used for grading.
- If your last submission does not compile or has a runtime error, then you will automatically receive a grade of 0. This can happen if you misspell the name of a required file or function or if your code has a bug in it.

Since we grade only your latest submission, you must ensure that your latest submission compiles!

- If you are submitting for the first time on Ed on the day the assignment is due, then you have waited too long.

Grading & Rubric

Test cases

The test cases total 100 points. To get full marks, you must pass all the test cases. Some tests are given to you (public), and you will receive feedback on whether or not you passed them, but you won't know if you passed the private tests until we grade your assignment.

Style marks

You must satisfy various style conditions. Style points will be subtracted from your test case grade as follows:

- **-1: complicated and/or unreadable expressions**
- **-1: very long lines**

One should not have to scroll horizontally to read the code. The commonly recommended maximum line length in Python is 79 characters, according to PEP 8, which is the Python Enhancement Proposal that provides the official style guide for writing Python code.

- **-1: too many or too few comments**

Place comments sparingly, e.g. when there is a complicated piece of logic. This will help you when you take a break from your code and return to it later. It will also help the T.A. if they need to understand your code.

Don't put comments for things that are obvious e.g. explain basic features of the Python language.

- **-1: unnecessary global variables**
- **-1: long or unnecessary (chain of) conditional statements.**

-
- **-1: comparing boolean expressions to True or False**
 - **-1: redundant variables**

Don't introduce variables that aren't needed, unless they represent a computed value that should be named so that the code is more clear.

- **-1: too much vertical spacing**

We will only take the point off in extreme cases for this and the following one

- **-1: no vertical spacing where it would be helpful**

We will also take points off for the following issues:

- **Up to -20: used material not seen in videos or in class activities unless explicitly authorized**
- **-15: submission had to be manually fixed and re-uploaded to correct a serious error**

Think of this as an admin fee.

- **-5: otherwise not following assignment instructions**